```
import { one, q } from '../../src/db.js';

// Postgres hands back a Date, pg-mem hands back a Date, a JSON body hands back
// a string. One shape from here on.
const isoDate = (v) => v instanceof Date
  ? v.toISOString().slice(0, 10)
  : String(v ?? '').slice(0, 10);

export const capabilities = [
  {
    key: 'availability.define',
    summary: 'Publish a bookable slot.',
    route: 'internal',
    handler: async ({ input, gateway }) => gateway.insert('slot', {
      resource: input.resource, on_date: input.date, starts: input.starts ?? null
      ends: input.ends ?? null, capacity: input.capacity ?? 1, price: input.price
      source: input.source ?? 'owner',
    }),
    verify: async ({ result, gateway }) => {
      const [row] = await gateway.select('slot', { where: { id: result.id } });
      return { state: row ? 'verified' : 'failed', evidence: [{ kind: 'row', id:
    },
  },

  {
    // The visitor never leaves the page, so the business keeps the thing the
    // booking platform normally keeps: what was asked for, and whether it existe
    key: 'availability.search',
    summary: 'Find open slots and record what the visitor was looking for.',
    route: 'internal',
    agentSafe: true,
    handler: async ({ ctx, input, gateway }) => {
      const rows = await gateway.select('slot', {});
      const open = rows.filter(r =>
        (!input.date || isoDate(r.on_date) === input.date) &&
        (!input.resource || r.resource === input.resource) &&
        (r.capacity - r.held) >= (input.partySize ?? 1)
      );

      await one(
        `insert into search_intent (business_id, surface, requested_for, party_si
         values ($1,$2,$3,$4,$5,$6) returning *`,
        [ctx.businessId, input.surface ?? 'links', input.date ?? null,
         input.partySize ?? null, input.resource ?? null, open.length]
```

```
    );

    return {
      matched: open.length,
      slots: open.map(r => ({
        id: r.id, resource: r.resource, date: isoDate(r.on_date),
        starts: r.starts, ends: r.ends, price: r.price,
        remaining: r.capacity - r.held,
      })),
    };
  },
  verify: async ({ result }) => ({
    state: 'verified', evidence: [{ kind: 'match_count', matched: result.matche
  }),
},

{
  key: 'availability.hold',
  summary: 'Hold a seat on a slot.',
  route: 'internal',
  handler: async ({ ctx, input, gateway }) => {
    const [slot] = await gateway.select('slot', { where: { id: input.slotId } }
    if (!slot) throw new Error('no such slot');
    const size = input.partySize ?? 1;
    if (slot.capacity - slot.held < size) throw new Error('not enough left on t
    const updated = await gateway.update('slot', slot.id, { held: slot.held + s
    // Mark the search that actually led here, not whatever was most recent.
    const hit = await one(
      `select id from search_intent
        where business_id = $1 and matched > 0 and requested_for = $2
        order by created_at desc limit 1`,
      [ctx.businessId, isoDate(slot.on_date)]
    );
    if (hit) await q(`update search_intent set converted = true where id = $1`,
    await gateway.emit('held', { slotId: slot.id, partySize: size });
    return updated;
  },
  verify: async ({ input, gateway }) => {
    const [row] = await gateway.select('slot', { where: { id: input.slotId } })
    return {
      state: row && row.held > 0 ? 'verified' : 'failed',
      evidence: [{ kind: 'read_back', held: row?.held ?? null, capacity: row?.c
    };
  },
},

{
```

```
    key: 'availability.release',
    summary: 'Give a held seat back.',
    route: 'internal',
    handler: async ({ input, gateway }) => {
      const [slot] = await gateway.select('slot', { where: { id: input.slotId } }
      return gateway.update('slot', slot.id, { held: Math.max(0, slot.held - (inp
    },
    verify: async ({ input, gateway }) => {
      const [row] = await gateway.select('slot', { where: { id: input.slotId } })
      return { state: row ? 'verified' : 'failed', evidence: [{ kind: 'read_back'
    },
  },
];

export const renderers = {
  // Whatever the booking backend is, it comes out in one shape here.
  calendar: async ({ gateway }) => {
    const rows = await gateway.select('slot', { orderBy: 'on_date' });
    const byDate = {};
    for (const r of rows) {
      const d = isoDate(r.on_date);
      (byDate[d] ??= []).push({
        id: r.id, resource: r.resource, starts: r.starts, ends: r.ends,
        price: r.price, remaining: r.capacity - r.held, source: r.source,
      });
    }
    return {
      bookable: true,
      days: Object.entries(byDate).map(([date, slots]) => ({
        date, open: slots.filter(s => s.remaining > 0).length, slots,
      })),
    };
  },
};
```